

Projeto Final – Detalhamento Checkpoints

1 Introdução

O curso de PDS II é orientado por um projeto de desenvolvimento de software. O projeto é realizado em grupos de até quatro alunos. O grupo deve escolher um problema de seu interesse e realizar todo o processo de desenvolvimento (análise, projeto e implementação) de uma solução. Espera-se ao final um sistema de porte médio (> 2000 linhas de código) que utiliza os conceitos e técnicas vistos durante o curso (modelagem, POO, testes unitários, etc). O programa deve ser feito baseado na linguagem C++11. O desenvolvimento e a entrega deverão ser feitos utilizando o sistema de controle de versão GitHub¹. Sugere-se que commits/pushs sejam feitos de maneira frequente, sempre que houver alterações.

2 Checkpoints

O projeto será desenvolvido de forma **incremental**, possibilitando a evolução contínua da solução ao longo do semestre. O acompanhamento do trabalho ocorrerá por meio de **10 checkpoints**, distribuídos ao longo da disciplina, cada um associado a entregáveis específicos que permitem monitorar o progresso e realizar ajustes quando necessário. Os detalhes de cada checkpoint encontram-se descritos a seguir e o cronograma está presente na figura 1. O não cumprimento de um checkpoint acarretará **penalização de 5% na nota final**, cumulativa a cada etapa não atendida.

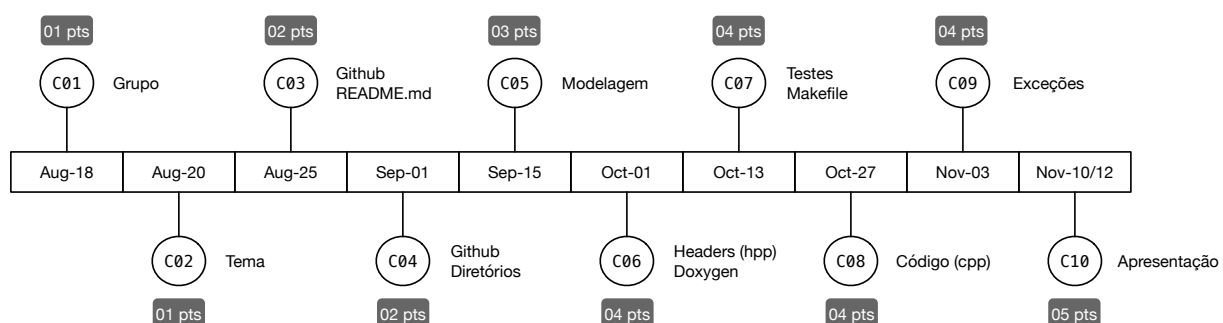


Figura 1: Checkpoints

C1. Formação do Grupo e Configuração do ambiente de desenvolvimento

Alunos devem indicar o grupo ao qual pertencem preenchendo um documento compartilhado, cujo link está disponível no Moodle. Cada grupo deve preparar e validar o ambiente

¹<https://github.com/>

de desenvolvimento, garantindo que todos os integrantes tenham instalados e configurados os compiladores, bibliotecas, IDEs e demais ferramentas necessárias. O objetivo é assegurar que não haverá obstáculos técnicos para o início do projeto.

C2. Definição do tema

Os grupos deverão escolher o tema do projeto e informar na Planilha do Moodle.

C3. Acesso ao GitHub e Commit de teste

Cada integrante do grupo deverá acessar o repositório no Github e realizar um commit de teste. Primeiro faça a correta configuração local de seu usuário, considerando:

```
git config user.name "O SEU NOME COMO ESTA NO MOODLE"  
git config user.email "O SEU EMAIL COMO ESTA NO MOODLE"
```

Em seguida, criar um arquivo de texto na raiz do projeto com nome **[sua matrícula].txt**, e adicione o seguinte conteúdo no arquivo.

Estou ciente que a avaliação da participação final será baseada única e exclusivamente na quantidade de commits realizada por mim. Entendo que não serão contabilizados commits feitos em desacordo com os dados de usuário cadastrados acima. A correta divisão de tarefas entre os membros é responsabilidade minha e de meu grupo.

C4. Estrutura de diretórios

O grupo deve organizar a estrutura inicial de diretórios do repositório, incluindo pelo menos as pastas **src/**, **include/**, **build/**, **tests/** e **design/**. O objetivo é adotar uma padronização que facilite a modularização do projeto.

C5. Requisitos e Modelagem

Nesta etapa, o grupo deve levantar os requisitos por meio de User Stories e elaborar a modelagem inicial utilizando Cartões CRC. Esses artefatos devem ser armazenados no diretório **design/**.

As User Stories² são uma forma simples de apresentar os requisitos funcionais desejados para um determinado sistema. São artefatos de desenvolvimento utilizados principalmente em processos baseados em metodologias ágeis. As descrições são intencionalmente genéricas, dando liberdade ao grupo para decidir detalhes da implementação.

O grupo deve identificar funcionalidades interessantes para o sistema e propor pelo menos **SEIS** User Stories. Para cada uma, deverá ser feita uma descrição sucinta, similar ao exemplo abaixo.

²https://en.wikipedia.org/wiki/User_story

- **Sistema de Gestão Acadêmica**

- **Descrição:**

- Como professor quero gerar um relatório para verificar o desempenho dos alunos.

- **CrITÉrios de aceitação:**

- Mostrar a pontuação da avaliação atual de um aluno.
- Mostrar a pontuação de avaliação passada de um aluno.
- Mostrar a média e desvio padrão da turma para a avaliação.

Tente apresentar User Stories para diferentes tipos de usuário do sistema (ou considerando papéis específicos em certos contextos). Cada User Story deve apresentar entre 3 e 5 critérios de aceitação.

Em seguida, faça pelo menos **SEIS** cartões CRC³ para as classes que você julga como sendo as mais importantes no sistema. Cada cartão deve apresentar entre 5 e 10 responsabilidades (considerando conhecimento e realização) e mencionar pelo menos 2 colaborações. Você pode usar esse site para lhe auxiliar a fazer o cartão: <https://h3ct0r.github.io/crcmaker/>

C6. Definição dos contratos (.hpp) e documentação inicial (Doxygen+README)

Os arquivos de cabeçalho devem ser definidos com as assinaturas das funções e classes que comporão o sistema, baseados na modelagem feita na etapa anterior. Paralelamente, deve-se iniciar a documentação usando o Doxygen⁴, descrevendo a estrutura proposta e funcionalidade dos métodos.

Além disso, o arquivo README⁵ do repositório deve ser atualizado com informações básicas, como o nome dos integrantes do grupo, a descrição inicial do problema a ser tratado, seus objetivos principais e a motivação para a escolha (1 a 2 parágrafos), e demais dados relevantes para contextualização do projeto.

C7. Testes de unidade (TDD) e Makefile

O grupo deve começar a implementar testes de unidade para o sistema seguindo a abordagem Test-Driven Development (TDD⁶). Nesse momento, é esperado que os testes ainda não passem, já que os comportamentos ainda não foram implementados. O objetivo é estruturar o processo de verificação futura do código. O grupo deverá utilizar o framework doctest⁷ para implementar os testes de unidade. Deve haver pelo menos uma classe de

³https://en.wikipedia.org/wiki/Class-responsibility-collaboration_card

⁴<http://www.doxygen.org/>

⁵<https://docs.github.com/pt/repositories/managing-your-repositorys-settings-and-features/customizing-your-repository/about-readmes>

⁶https://en.wikipedia.org/wiki/Test-driven_development

⁷<https://github.com/onqtam/doctest>

testes para cada uma das principais classes do sistema (por exemplo, as classes de entidades). Além disso, ao final, deve-se apresentar uma cobertura total do código de pelo menos 60% (a ser verificado utilizando a ferramenta gcovr⁸).

Também deve ser criado um Makefile para automatizar a compilação, garantindo reprodutibilidade.

C8. Implementação inicial dos comportamentos (.cpp)

As primeiras versões das funcionalidades devem ser implementadas nos arquivos .cpp, em conformidade com os contratos definidos nos cabeçalhos. O objetivo é validar a integração entre os módulos e iniciar a construção efetiva da solução.

C9. Programação defensiva e Tratamento de exceções

O código deve ser refinado para incluir práticas de programação defensiva, validando entradas, prevenindo erros comuns e implementando tratamento adequado de exceções. Essa etapa busca aumentar a robustez e a confiabilidade do sistema.

A interface de interação com a aplicação poderá ser feita via terminal de comando. Possíveis arquivos necessários durante a etapa de inicialização do sistema deverão ser fornecidos pelo grupo.

C10. Finalização e Apresentação

O grupo deve consolidar a versão final do projeto, realizando ajustes, revisões de código e testes finais. O grupo deve preparar uma apresentação para a turma. Essa apresentação pode durar até 16 minutos. A apresentação deve estar dividida em três partes:

- (4 minutos)** Contexto e problema resolvido: os alunos devem suar slides para descrever o projeto.
- (3 minutos)** Estrutura do repositório: os alunos devem mostrar o repositório, descrevendo sua estrutura e organização.
- (9 minutos)** Demonstração: os alunos devem mostrar, em uma sessão *live*, como o projeto desenvolvido pode ser usado.

3 Temas

Nesta seção são apresentadas algumas sugestões de temas para o projeto da disciplina. Para cada proposta, são descritas funcionalidades em alto nível, com o objetivo de orientar a modelagem inicial do sistema. Essas funcionalidades não devem ser encaradas como uma lista fechada de requisitos: os alunos podem (e devem) propor extensões, melhorias e novas funcionalidades, desde que coerentes com o escopo do projeto.

⁸<https://gcovr.com/en/stable/>

O sistema deve permitir a aplicação dos principais conceitos da disciplina, especialmente Programação Orientada a Objetos (encapsulamento, herança e polimorfismo), além de boas práticas, modularidade, tratamento de exceções e testes unitários, apresentando modelagem consistente e responsabilidades bem definidas. O objetivo não é apenas criar um programa funcional, mas um sistema confiável, reutilizável e de fácil manutenção e extensão, sendo valorizadas a aplicação adequada dos conceitos, a criatividade e a implementação de funcionalidades adicionais.

Por fim, recomenda-se a utilização de arquivos de texto para persistência de dados (por exemplo, armazenamento de cadastros, histórico de operações, configurações, etc.) ou para simulação de entradas do sistema. O uso de persistência simples permite exercitar manipulação de arquivos, organização de dados e tratamento de erros, sem introduzir complexidade adicional como bancos de dados externos.

- **E-commerce:** O desafio é desenvolver uma plataforma de comércio eletrônico capaz de gerenciar usuários, produtos e pedidos de forma organizada e segura, simulando o funcionamento básico de uma loja online.
 - Funcionalidades básicas: gestão de perfil de usuários, cadastro/listagem de produtos, carrinho de compras, efetivação e histórico de pedidos, relatórios/histórico, etc.
- **Mini rede social:** O desafio é construir uma mini rede social que permita modelar interações entre usuários e organizar conteúdos publicados em uma timeline.
 - Funcionalidades básicas: gestão de perfil de usuários, diferentes tipos de posts (e.g., texto, vídeo), listagem de posts em timeline, likes/comentários em posts, adicionar/listar amigos, etc.
- **Estação Espacial:** O desafio é desenvolver um sistema capaz de simular a gestão integrada de uma estação espacial, controlando módulos, tripulação e recursos críticos.
 - Funcionalidades básicas: diferentes tipos de módulos (e.g., habitação, laboratório), cadastro de astronautas e alocação de módulos, cadastro e gestão de recursos (e.g., oxigênio, energia, água), simulação e detecção de falhas em módulos, geração de relatórios de status da estação, etc.
- **Sistema de Presença / “Ponto Eletrônico” de Alunos:** O desafio é construir um sistema para o registro da presença dos alunos em diferentes matérias, departamentos, etc. de forma automatizada, utilizando o “identificador da carteirinha” (número hexadecimal de 48-bits), simulando o funcionamento de um sistema real de controle de acesso.
 - Funcionalidades básicas: Cadastro de diferentes tipos de alunos (e.g, Graduação, Mestrado, Isolada, etc.), em diferentes tipos de curso (e.g, graduação, pós-graduação), ter uma interface para validação da presença/carteirinhas dos alunos para cada dia de aula (modo aluno), uma interface para registro e edição dos alunos, cursos, etc (modo professor). Geração de relatórios (faltas por dia, por turma, por aluno, etc.).

- **Estacionamento Inteligente:** O desafio é construir um sistema que simule o funcionamento de um estacionamento inteligente, controlando vagas, veículos e regras de cobrança.
 - Funcionalidades básicas: cadastro de diferentes tipos de vagas (e.g., Comum, Especial) e veículos (e.g., Carro, Moto, Caminhão), simular entrada/saída de veículos, calcular diferentes tipos de tarifas (considerar veículo, vaga, tempo), controle e alocação de vagas, geração de relatórios, etc.
- **Jogo RPG (Turn-Based Game Engine Simplificado):** O desafio é implementar um motor simplificado de RPG baseado em turnos, modelando personagens, regras de combate e gerenciamento de recursos.
 - Funcionalidades básicas: diferentes tipos de personagens, combate por turnos, regras de dano específicas por classe, inventário, gestão e uso de itens, etc.
- **Engine para Jogos de Tabuleiro:** O desafio é desenvolver uma engine genérica para jogos de tabuleiro, capaz de fornecer a infraestrutura básica para criação de diferentes jogos (por exemplo, damas, batalha naval, etc.), separando regras específicas da lógica geral do sistema.
 - Funcionalidades básicas: representação do tabuleiro, representação de peças e jogadores, sistema de turnos, estrutura para definição de regras específicas de cada jogo, mostrar ao menos dois jogos distintos feitos utilizando a mesma engine.

4 Critérios de Avaliação

A avaliação leva em consideração algumas observações:

- O trabalho deverá ser feito em grupos com 4 alunos (excepcionalmente, pode-se aceitar até três grupo de cinco alunos, caso existam alunos sem grupos).
- Trabalhos copiados não serão avaliados.
- A entrega final será considerada o último commit na branch principal do projeto.
- Deve ser fornecido com o código um arquivo `Makefile` com as opções `make` e `make run`.

A distribuição de pontos é vista na Figura 1 deste documento. A pontuação individual de cada aluno é dada pela rotina apresentada na Figura 2.

A pontuação de cada checkpoint (Figura 1) é atribuída ao grupo como um todo, refletindo o trabalho coletivo de análise, modelagem e implementação. A partir dessa pontuação total do grupo, a nota individual de cada aluno é obtida pela ponderação de dois fatores:

1. **Participação mínima.** Espera-se que cada integrante contribua de forma ativa e frequente ao longo do projeto. Diferentemente de uma contagem bruta de commits, a participação é avaliada como um critério binário, exigindo que o aluno atinja simultaneamente um número mínimo de commits, de linhas efetivamente alteradas e

```
bool cumpriuParticipacaoMinima(  
    int commitsAluno, int linhasAluno, int diasAtivosAluno,  
    int minCommits = 8, int minLinhas = 100,  
    int minDiasAtivos = 4) {  
  
    return commitsAluno >= minCommits &&  
        linhasAluno >= minLinhas &&  
        diasAtivosAluno >= minDiasAtivos;  
}  
  
double notaFinalAluno(  
    double pontuacaoTotalGrupo, int commitsAluno,  
    int linhasAluno, int diasAtivosAluno,  
    double notaApresentacao) {  
  
    double fatorCommits = cumpriuParticipacaoMinima(  
        commitsAluno, linhasAluno, diasAtivosAluno) ? 1.0 : 0.0;  
  
    return pontuacaoTotalGrupo * fatorCommits * notaApresentacao;  
}
```

Figura 2: Cálculo da pontuação individual do aluno.

de dias distintos com atividade no repositório (métricas descritas adiante e extraídas pelo script da Figura 3). Um aluno que não atinja algum desses patamares recebe nota final zero, independentemente do desempenho do grupo; os valores mínimos exatos de cada métrica serão informados oportunamente pelos professores e monitores da disciplina.

2. **Desempenho na apresentação.** Na etapa C10, cada aluno deverá responder a uma pergunta sobre o sistema desenvolvido, recebendo uma nota entre 0 e 1 que reflete seu entendimento do projeto como um todo. Essa nota atua como fator multiplicativo sobre a pontuação obtida no critério anterior, penalizando alunos que não demonstrem conhecimento suficiente do sistema independentemente de sua contribuição ao repositório.

Dessa forma, a nota final de um aluno nunca ultrapassa a pontuação total conquistada pelo grupo, e reflete tanto o cumprimento de um patamar mínimo de participação efetiva no desenvolvimento quanto seu domínio sobre a solução apresentada.

4.1 Participação Ativa

Para mitigar o risco de que a contagem de commits seja manipulada por meio de alterações triviais (por exemplo, commits de correção de espaços em branco ou divisão artificial de uma única mudança em múltiplos commits), a participação de cada aluno não será medida apenas pelo número bruto de commits. Serão também consideradas duas métricas complementares, extraídas automaticamente do histórico do Git: o número de *linhas efetivas* alteradas pelo aluno (soma de inserções e remoções reportadas por `git log -numstat`,

desconsiderando *merge commits*) e o número de *dias ativos* distintos em que o aluno realizou ao menos um commit. Essas três métricas, calculadas por meio do script apresentado na Figura 3, precisam ser simultaneamente atingidas para que a participação do aluno seja considerada satisfatória, evitando que contribuições concentradas em rajadas de commits triviais próximas à data de entrega sejam suficientes para cumprir o critério. Os grupos devem estar cientes de que fluxos de trabalho que ocultam a autoria individual dos commits (como o *squash merge* de *pull requests*) podem prejudicar a correta contabilização da participação de cada membro, e portanto devem ser evitados.

```
#!/bin/bash
# uso: ./stats.sh <email_aluno>
AUTOR="$1"

COMMITTS=$(git log --no-merges --author="$AUTOR" \
  --oneline | wc -l)

LINHAS=$(git log --no-merges --author="$AUTOR" \
  --pretty=tformat: --numstat \
  | awk '{ins+=$1;_del+=$2}_END_{print_ins+del}')

DIAS=$(git log --no-merges --author="$AUTOR" \
  --pretty=format:@"%ad" --date=short \
  | sort -u | wc -l)

echo "Autor:_$AUTOR_|_Commits:_$COMMITTS_|_\
Linhas:_$LINHAS_|_Dias_ativos:_$DIAS"
```

Figura 3: Script para extração automática das métricas de participação (commits, linhas efetivas e dias ativos) a partir do histórico do Git.

5 Considerações Finais

O projeto final é um dos componentes mais importantes da disciplina, pois representa a oportunidade de aplicar, de forma integrada e prática, os conceitos discutidos ao longo do semestre: modelagem orientada a objetos, boas práticas de engenharia de software, testes automatizados e uso de ferramentas de controle de versão. Mais do que cumprir os checkpoints, espera-se que o processo de desenvolvimento contribua diretamente para a formação dos alunos como futuros profissionais da área.

Em caso de dúvidas técnicas ou dificuldades ao longo do desenvolvimento, os alunos devem procurar os monitores da disciplina, que estarão disponíveis para auxiliar na resolução de problemas e no esclarecimento de conceitos. É também natural e incentivado que os alunos troquem ideias com colegas de outros grupos sobre abordagens, dificuldades e soluções de projeto. Essa troca faz parte do processo de aprendizado e da cultura de desenvolvimento colaborativo. Ainda assim, é fundamental que cada grupo desenvolva sua solução por meios próprios. Trocar ideias é diferente de compartilhar código ou reproduzir soluções de terceiros: o valor do projeto está justamente no processo de análise,

decisão e implementação realizado pelo próprio grupo. Reforça-se, portanto, que trabalhos copiados, total ou parcialmente, não serão avaliados, conforme já indicado na Seção 4.